Security Review Report
NM-0965 - ENS V2 Additions



NETHERMIND
SECURITY

(October 19, 2026)

Contents

1 Executive Summary **2**

2 Audited Files **3**

3 Summary of Issues **4**

4 Changes Under Review **5**
4.1 Token Transfer and Emancipation . . . 5
4.2 Access Control . . . 5
4.3 Resolver Record Model . . . 6
4.4 Name Normalization and Universal Resolution . . . 6
4.5 Hosted Custody Accounts . . . 6
4.6 Migration and Supporting Contracts . . . 6

5 Risk Rating Methodology **7**

6 Issues **8**
6.1 [High] A malicious batch gateway can forge CCIP-Read callbacks and spoof resolution results . . . 8
6.2 [Medium] A session can assign an unauthorized subregistry when registering a name . . . 8
6.3 [Low] Documented migration order does not provide the ownership required by ETHRenewerV1 renewals . . . 9
6.4 [Low] Funding claim policy does not relate the destination delivery to the source amount it releases . . . 10
6.5 [Low] Wallet root resolver roles are unverified when a session reuses a resolver . . . 11
6.6 [Low] Writing one record field clears every other value a name inherits from the default . . . 11
6.7 [Info] Registrar approvals are not amount-bound and can survive session revocation . . . 12

7 Documentation Evaluation **14**

8 Test Suite Evaluation **15**

9 About Nethermind **16**

1 Executive Summary

This document presents the results of the security review conducted by Nethermind Security for the Solidity changes introduced by ENS contracts-v2 PR #417. The Ethereum Name Service (ENS) maps human-readable names to addresses and other records, while ENS V2 extends this model through hierarchical registries, role-based permissions, and compatibility paths for unmigrated ENS V1 names. The engagement reviewed the pull-request changes rather than re-auditing the complete ENS V2 codebase, using `b163b88` as the primary snapshot. The change set covered registries and resolvers, `.eth` registration, utilities, DNS and Universal Resolver integrations, reverse-registrar and migration components, and the new HCA execution-account subsystem. HCA received a dedicated review and was revalidated after its subsequent refactor at `c8a3166`. The review focused on authorization and role propagation, name ownership and record integrity, registration and renewal behavior, HCA trust boundaries, and offchain resolution safety.

The audit comprises 7778 lines of Solidity code. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report seven points of attention, where one is classified as `High`, one as `Medium`, four as `Low`, and one as `Informational`. The issue is summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

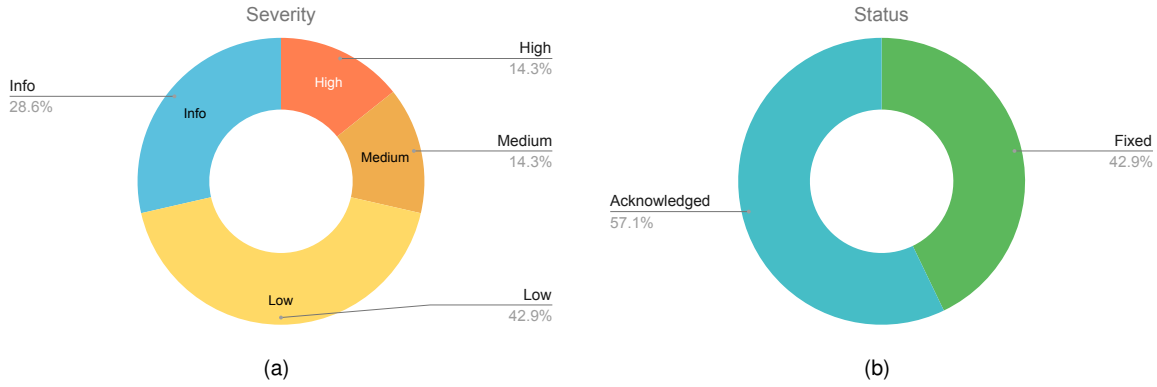


Fig. 1: Distribution of issues: Critical (0), **High** (1), **Medium** (1), **Low** (4), **Undetermined** (0), **Informational** (1), **Best Practices** (0). **Distribution of status: Fixed** (3), **Acknowledged** (4), **Mitigated** (0), **Unresolved** (0).

Summary of the Audit

Audit Type	Security Review
Initial Report	September 15, 2026
Final Report	October 19, 2026
Initial Commit	`b163b88`
Final Commit	`81869c3`
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

The table lists every Solidity file under `contracts/src/` that changed between 48b3e2d and 07453da. The **Changes** column gives lines added and removed over that range; the remaining columns describe each file as it stands at the final commit. Files marked `new` did not exist at the initial commit.

	Contract	Changes	LoC	Comments	Ratio	Blank	Total
1	src/access-control/EnhancedAccessControl.sol	+16 / -6	275	163	59%	49	487
2	src/access-control/interfaces/IEACGrantInitializable.sol	new +16	8	6	75%	3	17
3	src/access-control/interfaces/IEnhancedAccessControl.sol	+18 / -6	42	82	195%	25	149
4	src/access-control/libraries/EACBaseRolesLib.sol	+2 / -1	24	26	108%	8	58
5	src/dns/DNSAliasResolver.sol	+5 / -4	80	52	65%	15	147
6	src/dns/DNSTLDResolver.sol	+21 / -11	281	118	42%	34	433
7	src/dns/DNSTXTResolver.sol	+1 / -15	129	66	51%	17	212
8	src/erc1155/ERC1155Singleton.sol	+52 / -33	224	143	64%	40	407
9	src/hca/HCAAuthorizer.sol	new +53	22	20	91%	12	54
10	src/hca/HCAFundingSessionValidator.sol	new +615	445	112	25%	59	616
11	src/hca/HCAOwnerAndSessionValidator.sol	new +1057	806	155	19%	97	1058
12	src/hca/HCAValidatorBase.sol	new +31	17	11	65%	4	32
13	src/hca/StandaloneHCAFactory.sol	new +168	91	47	52%	31	169
14	src/hca/StandaloneSingleOwnerHCA.sol	new +410	252	105	42%	54	411
15	src/hca/interfaces/IStandaloneHCAFactory.sol	new +52	16	28	175%	9	53
16	src/hca/interfaces/IStandaloneHCAOwner.sol	new +15	5	8	160%	3	16
17	src/hca/libraries/HCAExecutionLib.sol	new +64	35	24	69%	6	65
18	src/hca/libraries/HCAOperationHashLib.sol	new +179	105	48	46%	27	180
19	src/hca/libraries/HCAPermit2Lib.sol	new +155	102	36	35%	18	156
20	src/hca/libraries/HCARegistrarPolicyLib.sol	new +102	69	26	38%	8	103
21	src/hca/libraries/HCAResolverPolicyLib.sol	new +204	142	45	32%	18	205
22	src/hca/libraries/HCASignatureLib.sol	new +130	79	37	47%	15	131
23	src/hca/libraries/HCASmartSessionLib.sol	new +194	119	50	42%	26	195
24	src/migration/Graveyard.sol	+2 / -3	128	56	44%	21	205
25	src/migration/LockedMigrationController.sol	+1 / -1	73	31	42%	14	118
26	src/migration/LockedWrapperReceiver.sol	+4 / -2	159	59	37%	27	245
27	src/migration/MigrationHelper.sol	+14 / -4	140	47	34%	25	212
28	src/migration/UnlockedMigrationController.sol	+1 / -1	104	49	47%	15	168
29	src/registrar/AbstractETHRegistrar.sol	+41 / -16	102	41	40%	27	170
30	src/registrar/BatchRegistrar.sol	+1 / -1	35	26	74%	13	74
31	src/registrar/ETHRegistrar.sol	+1 / -1	208	64	31%	32	304
32	src/registrar/ETHRenewerV1.sol	+34 / -14	95	52	55%	24	171
33	src/registrar/StandardRentPriceOracle.sol	+2 / -4	243	124	51%	58	425
34	src/registrar/interfaces/IETHRenewer.sol	+17 / -6	29	46	159%	15	90
35	src/registrar/interfaces/IRentPriceOracle.sol	+6 / -1	25	29	116%	9	63
36	src/registrar/interfaces/IRentPriceOracleProvider.sol	new +12	5	5	100%	3	13
37	src/registry/PermissionedRegistry.sol	+63 / -25	489	143	29%	67	699
38	src/registry/UserRegistry.sol	+21 / -11	63	24	38%	11	98
39	src/registry/WrapperRegistry.sol	+31 / -15	221	70	32%	29	320
40	src/registry/interfaces/IPermissionedRegistry.sol	+35 / -2	39	54	138%	23	116
41	src/registry/interfaces/IStandardRegistry.sol	+2 / -4	26	46	177%	15	87
42	src/registry/interfaces/IWrapperRegistry.sol	+1 / -15	7	14	200%	7	28
43	src/registry/interfaces/IWrapperRegistryInitializable.sol	new +21	11	8	73%	3	22
44	src/registry/libraries/RegistryRolesLib.sol	+12 / -0	34	29	85%	14	77
45	src/resolver/AbstractMirrorResolver.sol	+1 / -2	47	22	47%	14	83
46	src/resolver/AbstractRecordResolver.sol	new +217	166	33	20%	19	218
47	src/resolver/ENSV1Resolver.sol	+5 / -3	19	16	84%	10	45
48	src/resolver/ENSV2Resolver.sol	+3 / -3	29	18	62%	11	58
49	src/resolver/PermissionedResolver.sol	+225/-646	256	98	38%	35	389
50	src/resolver/PublicResolverV2.sol	+4 / -4	110	63	57%	23	196
51	src/resolver/interfaces/IPermissionedResolver.sol	+41 / -46	15	30	200%	12	57
52	src/resolver/interfaces/IPermissionedResolverInitializable.sol	new +13	5	6	120%	3	14
53	src/resolver/interfaces/IRecordResolver.sol	new +55	27	19	70%	10	56
54	src/resolver/interfaces/setters/IABISetter.sol	new +32	6	20	333%	7	33
55	src/resolver/interfaces/setters/IAddressSetter.sol	new +34	7	21	300%	7	35
56	src/resolver/interfaces/setters/IContenthashSetter.sol	new +23	5	14	280%	5	24
57	src/resolver/interfaces/setters/IDataSetter.sol	new +26	5	17	340%	5	27
58	src/resolver/interfaces/setters/IInterfaceSetter.sol	new +29	9	16	178%	5	30

Continued on next page

	Contract	Changes	LoC	Comments	Ratio	Blank	Total
59	src/resolver/interfaces/setters/INameSetter.sol	new +23	5	14	280%	5	24
60	src/resolver/interfaces/setters/ITextSetter.sol	new +26	5	17	340%	5	27
61	src/resolver/libraries/PermissionedResolverLib.sol	+51 / -60	40	34	85%	16	90
62	src/reverse-registrar/DefaultReverseRegistrarAdapter.sol	+32 / -3	41	24	59%	12	77
63	src/reverse-registrar/L2ReverseRegistrar.sol	+1 / -1	193	122	63%	56	371
64	src/reverse-registrar/L2ReverseRegistrarWithMigration.sol	+1 / -1	26	24	92%	13	63
65	src/reverse-registrar/ReverseRegistrarAdapter.sol	+33 / -3	40	26	65%	12	78
66	src/reverse-registrar/interfaces/IDefaultReverseRegistrarAdapter.sol	new +24	9	11	122%	5	25
67	src/reverse-registrar/interfaces/IReverseRegistrarAdapter.sol	new +24	7	13	186%	5	25
68	src/reverse-registrar/libraries/AccountNamerLib.sol	+9 / -1	24	16	67%	8	48
69	src/testnet/MigrationCorpusFixtures.sol	new +425	163	201	123%	62	426
70	src/testnet/MigrationFixtureBatcher.sol	new +180	89	68	76%	24	181
71	src/testnet/TestnetV1PremigrationRegistrar.sol	+1 / -1	157	84	54%	47	288
72	src/universalResolver/AbstractNormalizedUniversalResolver.sol	new +494	400	59	15%	36	495
73	src/universalResolver/UniversalHelper.sol	new +106	46	45	98%	16	107
74	src/universalResolver/UniversalResolverV1.sol	new +58	29	19	66%	11	59
75	src/universalResolver/UniversalResolverV2.sol	+19 / -57	40	18	45%	12	70
76	src/universalResolver/UpgradableUniversalResolverProxy.sol	+1 / -1	88	55	62%	31	174
77	src/universalResolver/interfaces/IENSIP15.sol	new +22	5	13	260%	5	23
78	src/universalResolver/interfaces/INormalizedUniversalResolver.sol	new +68	23	36	157%	10	69
79	src/universalResolver/interfaces/INormalizedUniversalResolverExtended.sol	new +49	23	23	100%	4	50
80	src/universalResolver/interfaces/IUniversalResolverExtended.sol	new +78	37	33	89%	9	79
81	src/universalResolver/libraries/LibENSIP15.sol	new +48	37	8	22%	4	49
82	src/universalResolver/libraries/LibResolution.sol	new +280	194	71	37%	16	281
83	src/utils/ContractNamer.sol	+1 / -1	23	13	57%	9	45
84	src/utils/LabelStore.sol	+1 / -1	28	20	71%	14	62
85	src/utils/PermissionedAddressSet.sol	+11 / -27	37	18	49%	13	68
86	src/utils/interfaces/IPermissionedAddressSet.sol	new +40	11	19	173%	11	41
87	src/utils/interfaces/IUnsafeTransferable.sol	new +19	6	11	183%	3	20
	Total	**+6,715 / -1,063**	**8136**	**3933**	**48%**	**1670**	**13739**

Three files present at the initial commit were removed over the range and therefore carry no row above: `src/registry/ApprovedUpgradeGate.sol`, `src/universalResolver/interfaces/IUniversalResolverV2.sol`, `src/universalResolver/libraries/LibRegistry.sol`.

3 Summary of Issues

	Finding	Severity	Update
1	A malicious batch gateway can forge CCIP-Read callbacks and spoof resolution results	High	Fixed
2	A session can assign an unauthorized subregistry when registering a name	Medium	Fixed
3	Documented migration order does not provide the ownership required by ETHRenewerV1 renewals	Low	Fixed
4	Funding claim policy does not relate the destination delivery to the source amount it releases	Low	Acknowledged
5	Wallet root resolver roles are unverified when a session reuses a resolver	Low	Acknowledged
6	Writing one record field clears every other value a name inherits from the default	Low	Acknowledged
7	Registrar approvals are not amount-bound and can survive session revocation	Info	Acknowledged

4 Changes Under Review

This section describes what changed on the `post-audit-2` branch relative to the codebase covered by NM-0919 - ENS V2. It does not restate the protocol architecture, which that report already documents. Each subsection names the contracts involved, what moved, and the behaviour a reader needs in order to follow the issues in Section 6.

The branch carries 39 commits touching 90 Solidity files under `contracts/src/`. The largest single area is token transfer, which was reworked across three pull requests opened while this review was under way and merged into the branch before it closed. The resolver was rebuilt on a new record model, and universal resolution was refactored to separate name normalization from resolution.

4.1 Token Transfer and Emancipation

`PermissionedRegistry` previously exposed only the standard ERC-1155 transfer entry points, and a transfer required `ROLE_CAN_TRANSFER_ADMIN` on the sending address. Three changes reshaped that path.

First, a deliberately unchecked route was added through a new interface, `IUnsafeTransferable`:

```
function unsafeTransfer(address to, uint256 tokenId, bytes calldata data) external;
function unsafeBatchTransfer(address to, uint256[] calldata tokenIds, bytes calldata data) external;
```

Both resolve the sender through the registry's expiry-aware `ownerOf(...)`, apply the ERC-1155 operator-approval check, and then move the token without the safety conditions described below. They still require `ROLE_CAN_TRANSFER_ADMIN` on the token.

Second, the role lookup in the transfer hook was narrowed. It previously resolved through the effective-roles path, in which a grant held on the root satisfies a token-level requirement; it now reads the token resource directly, so root authority no longer stands in for a grant the token never received:

```
uint256 resource = getResource(tokenId);
if ((_getRoles(resource, from) & RegistryRolesLib.ROLE_CAN_TRANSFER_ADMIN) == 0) {
    revert TransferDisallowed(tokenId, from);
} else if (safe && !isOnlyAssignee(tokenId, EACBaseRolesLib.ALL_ROLES, from)) {
    revert TransferUnsafeWithMultipleAssignees(tokenId, from);
}
```

Third, the standard path gained a registry-wide precondition. A registry is **emancipated** when its root resource holds none of the roles that would let root reach into a token:

```
uint256 internal constant UNEMANCIPATED_ROLE_BITMAP =
    ROLE_SET_SUBREGISTRY | ROLE_SET_SUBREGISTRY_ADMIN |
    ROLE_SET_RESOLVER    | ROLE_SET_RESOLVER_ADMIN |
    ROLE_UNREGISTER      | ROLE_UNREGISTER_ADMIN |
    ROLE_UPGRADE         | ROLE_UPGRADE_ADMIN;

function isEmancipated() public view virtual returns (bool) {
    return (RegistryRolesLib.UNEMANCIPATED_ROLE_BITMAP &
        EACBaseRolesLib.fromCounts(roleCount(ROOT_RESOURCE))) == 0;
}
```

`safeTransferFrom(...)` and `safeBatchTransferFrom(...)` revert with `TransferUnsafeUntilRegistryIsEmancipated()` on a registry that fails this test. The property cannot be revoked once held, because granting any role in the bitmap requires the matching admin role, which the bitmap also contains. `WrapperRegistry` overrides the test to return `true` unconditionally.

`ERC1155Singleton` was refactored alongside this. The receiver, sender and approval guards were factored into `_checkReceiver(...)`, `_checkSender(...)` and `_checkApproval(...)`, and `_update(...)` gained a `safe` flag so the registry can tell a standard transfer from an unsafe one.

4.2 Access Control

`EnhancedAccessControl` gained one primitive, promoted onto `IEnhancedAccessControl`:

```
function isOnlyAssignee(uint256 resource, uint256 roleBitmap, address account)
    public view returns (bool)
{
    (uint256 counts, ) = getAssigneeCount(resource, roleBitmap);
    return counts != 0 && counts == (roles(resource, account) & roleBitmap);
}
```

It answers whether an account holds every assigned role on a resource and no other account holds any, which is what the transfer path uses to establish that a token carries no delegated authority. `EACBaseRolesLib.fromCounts(...)` converts the packed per-role assignee counters into a role bitmap, and `hasAssignees(...)` was moved off the virtual dispatch chain.

4.3 Resolver Record Model

`PermissionedResolver` was rebuilt around numbered records rather than per-name storage, with the record types factored into a new `AbstractRecordResolver` and a set of per-profile setter interfaces. Records are numbered from one; a name maps to a record identifier, and several names may share a single record.

A name with no record of its own resolves through the record linked at the root name, which the contract calls the default record:

```
function _record(bytes32 node) internal view override returns (Record storage) {
    uint256 recordId = _recordIds[node];
    if (recordId == 0) {
        recordId = _recordIds[bytes32(0)]; // use default
    }
    return _records[recordId];
}
```

Every setter routes through `_ensureRecord(...)`, which creates and links an empty record the first time any field is written for a name. The new record is not seeded from the default, so the fallback stops applying from that point. `linkToNode(...)` and `linkToRecord(...)` move a name between records under `ROLE_LINK`, and `linkToRecord(name, 0)` returns it to the default.

Setter permissions are scoped by argument rather than by name. `grantSetterRoles(...)` decodes a setter call and grants the matching role on a resource derived from the argument, so an account can hold `ROLE_SET_TEXT` for one key across every name.

4.4 Name Normalization and Universal Resolution

Universal resolution was split apart. `AbstractNormalizedUniversalResolver` now holds the resolution walk, ENSIP-15 normalization moved into `LibENSIP15` behind an `IENSIP15` interface, and resolution helpers moved into `LibResolution`. A `UniversalResolverV1` entry point was added beside `UniversalResolverV2`, and the interfaces were reorganized into `INormalizedUniversalResolver` and `IUniversalResolverExtended`. `LibRegistry` and `IUniversalResolverV2` were removed.

4.5 Hosted Custody Accounts

The contracts under `contracts/src/hca/` are new over this range, adding roughly 2,300 lines across six contracts and seven libraries. They provide an ERC-4337 account that can hold ENS names and register them on a user's behalf, built on Rhinestone's `Nexus` rather than on the registry stack described above, so the access-control model here is unrelated to `EnhancedAccessControl`.

`StandaloneSingleOwnerHCA` extends `Nexus` and fixes its owner during initialization. Module changes are disabled once initialized, so the validator configured at deployment stays the only validation path for the life of the account, and upgrades are owner-triggered but require approval of both the target implementation and its predecessor.

`StandaloneHCAFactory` governs which implementations may be deployed and records the binding between an account and its owner. Deployment is permissionless, and the factory certifies the owner it observes immediately after initialization, making that binding immutable thereafter. `HCAAuthorizer` is the shared helper other contracts use to authorize an account through that certified binding.

Two fixed validator modules sit on top of `HCAValidatorBase`, which supplies the stateless ERC-7579 module surface both share. The permission checks are written into these contracts rather than configured through dynamic policy modules, so the set of operations a session may perform is fixed at deployment:

- `HCAOwnerAndSessionValidator` validates owner signatures in Rhinestone's HCA format and authorizes scoped ENS sessions, which carry a reusable owner authorization and are consumed through the intent executor's ERC-1271 path. Session registrations are required to pass a zero subregistry; owner calls are not restricted by this policy;
- `HCAFundingSessionValidator` covers the source-chain half of a cross-chain registration. The session it installs can pull the owner's permitted token into the account and sign a Permit2 claim delivering the configured token to the configured account.

The supporting libraries carry the parts of that validation that are shared or delicate. `HCAOperationHashLib` and `HCASmartSessionLib` reconstruct the hashes that Rhinestone intents and Smart Sessions sign, `HCASignatureLib` recovers the signature variants the Rhinestone and Nexus integrations produce, and `HCAPermit2Lib` decodes and hashes the fixed Permit2 claim format. `HCARegistrarPolicyLib` identifies the registrars a session may call and the payment tokens their rent oracles accept, `HCAResolverPolicyLib` validates resolver calls, deployments and implementation bindings against the canonical permissioned-resolver interfaces, and `HCAExecutionLib` reads selectors and ABI words out of policy-controlled execution calldata.

4.6 Migration and Supporting Contracts

The migration path changed in smaller ways. `MigrationHelper` was renamed and given contract naming, and `LockedWrapperReceiver` continues to translate ENSv1 fuses into v2 roles, including the subregistry root bitmap that every `WrapperRegistry` receives at deployment. New testnet-only contracts, `TestnetV1PremigrationRegistrar`, `MigrationCorpusFixtures` and `MigrationFixtureBatcher`, support pre-migration rehearsal on test networks.

Elsewhere, the reverse registrar gained verifiable adapters, `StandardRentPriceOracle` moved behind an `IRentPriceOracleProvider` interface, and `ApprovedUpgradeGate` was removed in favour of the allowlist held by `WrapperRegistry` through its `UPGRADE_SET`.

5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [High] A malicious batch gateway can forge CCIP-Read callbacks and spoof resolution results

File(s): contracts/src/universalResolver/AbstractNormalizedUniversalResolver.sol

Description: The Universal Resolver finds the resolver responsible for an ENS name and asks it for a requested record, such as the address associated with the name. When the resolver needs offchain data, a batch gateway retrieves that data on its behalf. The gateway should only transport the response because the resolver's own callback is responsible for checking it before the result is returned.

When the batch gateway marks a single lookup as failed, `CCIPBatcher.ccipBatchCallback(...)` stores the gateway-provided failure bytes in the lookup and sets `FLAG_BATCH_ERROR`. `AbstractNormalizedUniversalResolver.resolveBatchCallback(...)` then reverts those bytes verbatim:

```
if ((lu.flags & FLAG_BATCH_ERROR) != 0) {
    assembly {
        revert(add(answer, 32), mload(answer))
    }
}
```

A malicious or compromised batch gateway can make these failure bytes look like another `OffchainLookup`. Unlike an ordinary error, `OffchainLookup` instructs the user's client to fetch data from specified URLs and pass the response to a specified callback. The gateway can point the client to an attacker-controlled URL and select the Universal Resolver's `resolveCallback(...)`, while supplying an attacker-selected resolver address as the callback context. The proxy rewrites the apparent sender to its own address, so the client accepts the forged instruction, and the original resolver's callback never gets an opportunity to check the result.

For example, a user may enter `merchant.eth` in a wallet to send 10,000 USDC. If the name uses an offchain resolver and the wallet performs the lookup through a compromised batch gateway, the gateway can make the Universal Resolver return the attacker's address instead of the merchant's address. The wallet then constructs a transfer to the attacker, which the user may sign believing that they are paying `merchant.eth`. The ENS record remains unchanged, and users resolving the same name through an honest gateway still receive the correct address, allowing the gateway to target individual requests without recording the spoofed result onchain.

Recommendation(s): Consider strictly validating and propagating only the `HttpError` and `Error(string)` formats permitted for failed batch requests, while wrapping or rejecting malformed, unknown, and OffchainLookup-encoded failure data.

Status: Fixed

Update from the client: Fixed in PR #445

6.2 [Medium] A session can assign an unauthorized subregistry when registering a name

File(s): contracts/src/hca/libraries/HCARegistrarPolicyLib.sol

Description: A standalone HCA is an owner-controlled account that uses a restricted session key to perform ENS registration flows without granting the key general authority over the account. When the session registers a name, the registration policy ensures that the name is assigned to the HCA owner and uses the resolver authorized by the owner. However, `HCARegistrarPolicyLib.registrationFields(...)` reads only the `owner` and `resolver` arguments from each `ETHRegistrar.register(...)` call:

```
function registrationFields(bytes memory callData)
    internal
    pure
    returns (address registrant, address resolver)
{
    registrant = HCAExecutionLib.readAddress(callData, 4 + 32);
    resolver = HCAExecutionLib.readAddress(callData, 4 + 128);
}
```

`ETHRegistrar.register(...)` accepts a `subregistry` argument, but the session policy does not inspect it. The commitment does not provide an independent restriction because the session can submit a commitment for the same attacker-selected value. The registrar then forwards `subregistry` unchanged to `PermissionedRegistry`:

```
function register(
    string calldata label,
    address owner,
    bytes32 secret,
    IRegistry subregistry,
    address resolver,
    uint64 duration,
```

```
    IERC20 paymentToken,
    bytes32 referrer
) external returns (uint256 tokenId) {
    _consumeCommitment(
        makeCommitment(label, owner, secret, subregistry, resolver, duration, referrer)
    );

    // ...

    tokenId = ETH_REGISTRY.register(
        label,
        owner,
        subregistry,
        resolver,
        REGISTRATION_ROLE_BITMAP,
        uint64(block.timestamp) + duration
    );
}
```

`PermissionedRegistry` stores this value as the registered name's child registry. A compromised or malicious session key can therefore register a name to the correct owner and resolver while assigning an attacker-controlled child registry, giving the attacker persistent control over the namespace below the newly registered name until the owner detects and replaces the subregistry. The owner receives the roles needed to make that replacement, so the namespace compromise is recoverable.

Recommendation(s): Consider requiring `subregistry == address(0)` or binding the permitted subregistry into the owner-signed session authorization.

Status: Fixed

Update from the client: Fixed in PR #443

6.3 [Low] Documented migration order does not provide the ownership required by ETHRenewerV1 renewals

File(s): contracts/src/registrar/ETHRenewerV1.sol

Description: The migration documentation authorizes `ETHRenewerV1` as a controller in phase 4 so that names remain renewable, but does not transfer ownership of the ENSv1 base registrar to it until phase 6. The relevant phase descriptions state:

```
Phase 4: authorize ETHRenewerV1

- **Expected outcome:** `ETHRenewerV1` authorized as a v1 `BaseRegistrar` controller; unmigrated names
  stay renewable, each renewal extending both the v1 registration and the v2 reservation in one
  transaction. This does **not** reopen the phase 3 registration freeze.

Phase 6: enable the v2 controller

- **Prerequisites:** phase 5 complete. Order matters — the handoff controllers must be authorized
  **before** `activate-v1-renewer` transfers v1 `BaseRegistrar` ownership away from the v1 owner
  (after which the v1 owner can no longer manage controllers).

- **Expected outcome:** `BatchRegistrar` roles revoked (pre-migration seeding ends); `Graveyard` (+
  testnet helper) authorized as v1 controllers; v1 `BaseRegistrar` ownership transferred to
  `ETHRenewerV1` (final lock-down); `ETHRegistrar` granted `REGISTRAR | RENEW` on the v2 `ETHRegistry`
  — live v2 registrations open.
```

Each renewal unconditionally calls `syncWrapper(...)`, which invokes the owner-only `addController(...)` and `removeController(...)` functions:

```
function renew(RenewData calldata rd, IERC20 paymentToken) public override {
    super.renew(rd, paymentToken);
    string[] memory labels = new string[](1);
    labels[0] = rd.label;
    syncWrapper(labels);
}

function syncWrapper(string[] memory labels) public {
    BASE_REGISTRAR.addController(address(NAME_WRAPPER));
    for (uint256 i; i < labels.length; ++i) {
        WRAPPED_CONTROLLER.renew(labels[i], 0);
    }
    BASE_REGISTRAR.removeController(address(NAME_WRAPPER));
}
```

The ENSv1 `BaseRegistrarImplementation` restricts controller management to its owner:

```
function addController(address controller) external override onlyOwner {}

function removeController(address controller) external override onlyOwner {}
```

During phases 4 and 5, the underlying renewal is authorized, but `addController(...)` reverts and rolls back the complete transaction. The path works after the phase 6 ownership handoff, or if all required controller grants and the ownership handoff are completed before renewals are enabled.

Recommendation(s): Consider aligning the migration scripts and documentation with the permissions required by the renewal path. This can be done by completing the required controller grants and transferring ownership before enabling renewals, or by deferring wrapper synchronization until `ETHRenewerV1` owns the base registrar.

Status: Fixed

Update from the client: Fixed in PR #427

6.4 [Low] Funding claim policy does not relate the destination delivery to the source amount it releases

File(s): contracts/src/hca/HCAFundingSessionValidator.sol

Description: A source funding session fixes a token pair, a destination recipient and chain, and a ceiling on each side. Its stated purpose is to limit how a source account may fund the HCA, so the bound the owner approves is meant to be the bound a session key can move.

`_decodeAndValidateClaim(...)` checks the two amounts, each against its own ceiling, and never against each other:

```
if (
    claim.sourceToken != config.sourceToken ||
    claim.sourceAmount == 0 ||
    // @audit-issue The source amount is capped from above but never related to what is delivered.
    claim.sourceAmount > config.maxSourceAmount
) {
    revert ClaimFieldMismatch(4);
}
// ....
if (
    claim.tokenOut != config.destinationToken ||
    claim.amountOut == 0 ||
    // @audit-issue The destination amount has an upper bound only, so a dust delivery passes.
    claim.amountOut > config.maxDestinationAmount
) {
    revert ClaimFieldMismatch(8);
}
```

Both bounds run in the same direction. They prevent over-delivery and leave under-delivery unconstrained, which is the direction that costs the owner. The mandate's quote word and destination-operations hash are passed opaquely into the digest reconstruction and are never policy-checked, so nothing ties a claim to the off-chain quote it was derived from.

The owner's authorization does not compensate, because it commits only to the static session fields and to neither amount in any individual claim:

A session key alone therefore selects both numbers. It can sign a claim releasing the full `maxSourceAmount` from the source account while promising a negligible `amountOut`, pull the owner's permitted balance in as the matching funding operation, and settle against that dust delivery as the filler. Permit2 nonces are unordered and no cumulative spend is tracked, so the pattern repeats with fresh nonces until the session expires or the owner's allowance is exhausted. This is a direct loss of owner funds rather than a misdirection, bounded per claim by `maxSourceAmount` and unbounded in aggregate across the session window.

Recommendation(s): Consider enforcing a relation between the two amounts rather than only a ceiling on each, for example a minimum destination delivery expressed as a rate or a floor that the owner authorizes alongside the existing caps, so that a claim releasing the full source budget cannot settle against a negligible delivery.

It would also be worth committing the quote to the authorization, or tracking cumulative spend across claims, since the session currently permits the maximum per-claim outflow to be repeated with fresh nonces for as long as the owner's allowance lasts.

Status: Acknowledged

Update from the client: The budget for HCA's spend comes from the user's approved funding, and we acknowledge a compromised session could spend the approved funding

6.5 [Low] Wallet root resolver roles are unverified when a session reuses a resolver

File(s): contracts/src/hca/HCAOwnerAndSessionValidator.sol, contracts/src/hca/libraries/HCAResolverPolicyLib.sol

Description: The documented session rule is that every registration gives the wallet root `ROLES.ALL` on the resolver, so the owner is always a co-administrator of the resolver their name points at.

That holds only when the batch also deploys the resolver, where `HCAResolverPolicyLib.checkDeployment(...)` requires exactly two grants, root `ALL_ROLES` to the account and to its owner. A registration that reuses an existing resolver takes a different path. The register branch checks the registrant and the session resolver and nothing about permissions:

```
if (selector == IETHRegistrar.register.selector) {
    if (!_isAuthorizedRegistrar(execution.target, state)) {
        revert ActionNotAllowed(execution.target, selector);
    }
    (address registrant, address resolver) = _registerFields(execution.callData);
    // @audit-issue The owner's roles on an existing resolver are never established.
    if (registrant != owner || resolver != allowedResolver) {
        revert PolicyRuleFailed();
    }
```

`checkBinding(...)` then confirms only that the resolver is a factory-verified proxy of the permitted implementation, which establishes what kind of contract it is and not who controls it:

```
// ....
try IVerifiableFactory(factory).verifyContract(resolver) returns (
    address verifiedImplementation
) {
    // @audit-issue Implementation identity is checked; role assignments are not.
    if (verifiedImplementation != implementation) {
        revert PolicyRuleFailed();
    }
} catch {
    revert PolicyRuleFailed();
}
```

A resolver deployed by some other account, with its grants pointing at that account, satisfies both checks. The resolver address enters the session salt, so the owner does sign it, but it reaches them as an opaque address selected by the application with nothing surfacing whose resolver it is. An application that is malicious or compromised can therefore bind the session to a resolver it administers.

The result is a registration that vests the name in the owner while pointing it at records the owner cannot write. Because `PermissionedResolver` authorizes by record key rather than by name, whoever holds root roles on that resolver controls the name's records, including its address. The owner receives `ROLE_SET_RESOLVER` at registration and can repoint the name to a resolver they control, so this is recoverable.

Recommendation(s): Consider checking that the owner holds root `ALL_ROLES` on any resolver a registration reuses, using the same standard the deployment path already applies, so the documented guarantee holds on both paths rather than only when the resolver is created in the batch.

One option short of that is to require the resolver to have been deployed by the account itself, which the counterfactual address derivation already makes cheap to verify and which would rule out binding a session to a resolver belonging to someone else.

Status: Acknowledged

Update from the client: We can change the doc on this, but the deployment check for owner is more of a check on intended behaviour than being a security boundary. If an app for whatever reason explicitly wanted to use a resolver with other config, the crucial part is just that the HCA has full perms over it.

6.6 [Low] Writing one record field clears every other value a name inherits from the default

File(s): contracts/src/resolver/PermissionedResolver.sol

Description: `PermissionedResolver` stores records as numbered objects rather than one per name. A name maps to a record identifier and several names can share a single record. A name that has no identifier of its own reads from the record registered against the root node, which the contract's own documentation calls the default record and describes as manageable through the root name `0x00`:

```
function _record(bytes32 node) internal view override returns (Record storage) {
    uint256 recordId = _recordIds[node];
    if (recordId == 0) {
        recordId = _recordIds[bytes32(0)]; // use default
    }
    return _records[recordId];
}
```

This makes the default a shared template. A deployment can point many names at it and have each of them resolve an address, a contenthash, a primary name, and text entries without holding storage of its own.

Every setter writes through `_ensureRecord`, which gives a name its own identifier the first time anything is written to it:

```
function _ensureRecord(bytes memory name) internal returns (uint256 recordId) {
    bytes32 node = NameCoder.namehash(name, 0);
    recordId = _recordIds[node];
    if (recordId == 0) {
        recordId = ++_recordCount;
        _recordIds[node] = recordId;
        emit Linked(recordId, node, name);
    }
}
```

The record it creates is empty and is not seeded from the default. Since `_record` consults the default only while a name has no identifier, the fallback stops applying the moment that identifier exists. A name that was reading an address, a contenthash, a primary name, and text entries from the default loses all of them as a side effect of the first write to any one field. Writing a single text entry causes `addr` to resolve as the zero address for that name.

The permission model widens who can trigger this. `grantSetterRoles` issues argument-scoped authority, so an account can hold `ROLE_SET_TEXT` for one specific key and nothing else, with each setter checking a resource derived from its own argument:

```
function setText(bytes calldata name, string calldata key, string calldata value)
    external
    onlyRoles(PermissionedResolverLib.resource(key), PermissionedResolverLib.ROLE_SET_TEXT)
{
    uint256 recordId = _ensureRecord(name);
    _records[recordId].texts[key] = value;
    emit TextUpdated(recordId, key, key, value);
}
```

Such an account cannot write any other field, yet its one permitted write clears the effective value of every profile the name was inheriting. The transition is also quiet. The only event emitted is `Linked`, which reads as internal bookkeeping rather than as notice that a name's address has just changed.

Recommendation(s): Consider seeding a newly created record from the default rather than from empty, so that a first write changes only the field it targets. If a clean break from the default is the intended semantic, it would be worth emitting an event that names the transition so that indexers and name holders can observe it, and requiring a root-level role such as `ROLE_LINK` to trigger it rather than allowing any argument-scoped setter to do so.

Status: Acknowledged

Update from the client: We claim this is expected behavior.

6.7 [Info] Registrar approvals are not amount-bound and can survive session revocation

File(s): contracts/src/hca/HCAOwnerAndSessionValidator.sol

Description: The destination session policy allows an ERC-20 approval when the spender currently holds the registry's root registrar role and the registrar's rent price oracle accepts the token. `_checkPaymentTokenApproval(...)` validates the spender and token but does not read the approval amount:

```
address spender = _readAddress(callData, 4);
if (
    _isAuthorizedRegistrar(spender, state) &&
    HCARegistrarPolicyLib.isPaymentToken(spender, token)
) {
    return;
}
```

The approval does not have to match a registration price or accompany a registration, so a session can call `approve(registrar, type(uint256).max)` as its entire operation. The reveal-batch instructions tell the frontend to approve only the current registration price, while the on-chain policy constrains only the spender and token. Registration quantity and spending from the HCA's residual balance are therefore delegated to the session key rather than limited to one owner-selected registration.

`revokeSessions()` increments the nonce used by the validator and prevents future session-validated operations, but it does not reverse ERC-20 approvals the session already executed. An allowance therefore remains after the session expires or is revoked and after the spender later loses the registrar role. The current `ETHRegistrar.register(...)` only pulls payment from `msg.sender`, so its standing allowance cannot independently remove funds from the HCA after revocation; direct use of the surviving allowance would additionally require a malicious or compromised approved registrar. The behavior is recorded as a clarification of the session, registrar, and ERC-20 trust boundaries rather than an implementation vulnerability.

Recommendation(s): Consider clarifying that “approve only the current registration price” is a frontend construction requirement rather than an invariant enforced by the session validator, and that session revocation does not clear existing ERC-20 allowances. If approvals are intended to be constrained on-chain, consider binding them to the aggregate registration price in the operation or to an owner-signed session spending limit.

Status: Acknowledged

Update from the client: We trust the registrar to spend correctly.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about ENS's contracts documentation

For a differential review the decisive artefact is a description of what changed, and the ENS team supplied one. A per-file change map accompanied the branch, attributing each file to the sub-pull-request that touched it, ranking the changes by blast radius, and assigning each one a recovery-cost tier. That document set the review order and made it possible to separate mechanical refactors from behavioural changes without rediscovering the distinction from the diff. The repository also carries dedicated material on the migration and pre-migration procedures and on the hosted account component, and the contracts themselves are documented throughout with NatSpec, including header comments that explain the record model and the role layout rather than restating the function signatures.

The team was responsive throughout. Questions about intended behaviour were answered directly. Where a reported issue turned out to describe a deliberate design decision, the reasoning behind that decision was explained rather than merely asserted.

8 Test Suite Evaluation

The project uses Foundry. Running `forge test` at the final commit executes 1,069 tests across 48 suites, with no failures and none skipped. The suite is broad and, for the areas under review, specific: the registry tests exercise the role bitmap, the expiry and version lifecycle, and the transfer paths across both the standard and the unsafe entry points, and the migration tests build real ENSv1 wrapper states and drive them through the migration controllers rather than asserting against mocks.

The suite also tracked this review. Every issue reported against the transfer rework was accompanied, in the fix, by tests covering the same path, including the reserved and never-registered cases that the original change did not exercise. Several behaviours that might otherwise have been read as defects are pinned by tests that state the intended rule directly, which is what allowed two reported issues to be resolved as intended behaviour rather than left open.

Test Result	Passed	Failed	Skipped
`forge test`	1,069	0	0

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.